



Technical Report – Solar Tracker

T3 - Specialist Technician in Automation,
Robotics, and Industrial Control

Bruna Ferreira Gomes Madeira



1. Introduction

This mini-project consists of the design, development, and validation of a mechatronic system applied to the field of renewable energy, specifically a dual-axis solar tracker. The system is based on an Arduino platform and uses servomotors to dynamically position a photovoltaic module.

The main objective of the developed solution is to maximize solar energy capture throughout the day by continuously adjusting the panel orientation. Unlike approaches based exclusively on photosensitive sensors, the system relies on a solar positioning algorithm based on pre-collected data, ensuring greater stability and predictability.

Solar position data were obtained using the SunCalc tool and integrated into the control logic to determine the optimal azimuth and elevation angles.

2. System Components

The developed system integrates the following main components:

- Arduino Uno – central control unit
- RTC DS1302 module – real-time date and time acquisition
- 20x4 LCD display with I2C interface – user interaction interface
- Azimuth servomotor – horizontal axis control
- Tilt servomotor – vertical axis control
- LDR sensor – ambient light intensity measurement
- EEPROM memory (internal) – persistent data storage
- 5V power supply
- Dual-axis mechanical support structure

3. Location and Solar Data

The system was developed considering the following geographic coordinates:

- Latitude: 41.53773°
- Longitude: -8.43825°

Solar position data were obtained for Braga, Portugal, on April 16, 2026, with 15-minute intervals. These data include azimuth and solar altitude values, which are essential for determining panel orientation.

4. Development Stages

4.1 Solar Position Study



An initial analysis of the sun's apparent motion was conducted to understand the concepts of solar azimuth and elevation. This study enabled the definition of the operational angular range and servo mapping limits.

4.2 Manual Experimental Validation

A manual control system using potentiometers was implemented to validate the mechanical structure and movement ranges. A 16x2 LCD display was used for feedback, and the RTC module was configured during this stage.

4.3 Automatic System Integration

The integration between the RTC module and the solar position table allowed the system to automatically adjust the azimuth and tilt angles based on the current time.

During nighttime operation, the system moves to a predefined home position:

- Azimuth: 90°
- Tilt: 0°

4.4 Sensor Integration and Interface Enhancement

An LDR sensor was introduced to classify ambient light conditions into three states:

- SUN (high irradiance)
- PARTLY (moderate irradiance)
- CLOUDY (low irradiance)

The display was upgraded to a 20x4 LCD, enabling the visualization of:

- Current time
- System status
- Panel position
- Team identification

4.5 Data Persistence

EEPROM memory was used to store critical system parameters, ensuring data recovery after power loss. The system validates stored values and applies safe defaults when necessary.

5. Data Structure

A structured data type was defined to represent solar position:



```
typedef struct {  
    int hour;  
    int minute;  
    float azimuth;  
    float altitude;  
} SolarPosition;
```

This structure groups all relevant parameters into a single entity, improving code organization and maintainability.

The solar dataset is implemented as an array of this structure, covering the operational time range.

6. Control Algorithm

6.1 Execution Flow

The system operates in a continuous loop performing:

- LDR sensor reading
- Time acquisition via RTC
- Validation of operational time window
- Light condition classification
- Solar position determination
- Conversion to servo angles
- Actuator update
- Display update
- EEPROM storage

6.2 Value Conversion

Solar azimuth and altitude values are converted into servo angles using proportional mapping functions and constrained within safe operating limits.

6.3 Solar Position Selection

The system determines the most appropriate solar position using a nearest-time matching algorithm, ensuring continuous operation even without exact time matches.

7. Safety and Operational Limits



7.1 Angular Limits

- Azimuth: 0° to 180°
- Tilt: 0° to 75°

7.2 Night Operation

Outside the solar operation window, the system enters NIGHT mode and moves the panel to a safe home position.

7.3 Circuit Separation

Electrical separation between logic and power circuits was implemented to reduce noise and ensure stable I2C communication.

8. User Interface

The 20x4 LCD display provides real-time information distributed across four lines:

- System identification
- Current time
- System status
- Panel position

At startup, an initialization screen displays system and location information.

9. Testing and Validation

The system underwent several validation tests, including:

- Axis movement verification
- Mechanical stability assessment
- Position repeatability
- Night mode functionality
- Power failure recovery
- Light condition classification

Results confirm correct system operation.

9.1 Control Strategy Evaluation



The algorithm-based control approach presents significant advantages over purely sensor-based systems:

- Improved stability
- Reduced oscillations
- Predictable behavior

The hybrid approach (algorithm + sensor validation) enhances system robustness and efficiency.

10. Functional Architecture

The system architecture consists of the following functional blocks:

- RTC → time reference
- LDR → environmental sensing
- Arduino → processing and control
- Servomotors → mechanical actuation
- LCD → user interface
- EEPROM → data storage

11. Future Improvements

Planned system enhancements include:

- Migration to ESP32 platform
- Integration with Raspberry Pi
- Web-based user interface
- Energy monitoring dashboard
- Remote system monitoring
- Wind-safe parking mode
- Performance comparison with fixed panels
- Refinement of solar data across seasons

12. Technical Analysis of Code Excerpts

This section presents a technical analysis of the main code excerpts used in the project. Each part focuses on how specific components work, explaining their role and contribution to the system's overall functionality.



12.1 LDR Sensor Reading

The analysis of the `analogRead(pinLDR)` instruction focuses on its expected output range, hardware constraints, and physical interpretation.

The Arduino Uno analog-to-digital converter (ADC) provides 10-bit resolution, producing integer values between 0 and 1023, corresponding to input voltages from 0 V to 5 V.

The use of an analog input pin is required due to the continuous nature of the LDR output signal, which varies proportionally with incident light intensity. Digital inputs, limited to binary states, are not suitable for this application.

The obtained values are processed through averaging techniques to reduce electrical noise and are subsequently used for classification via threshold comparison.

High LDR readings represent strong solar irradiance conditions, corresponding to direct sunlight exposure.

12.2 Weather State Classification

The function responsible for evaluating environmental conditions classifies ambient light into three discrete states: **SUN**, **PARTLY**, and **CLOUDY**.

This classification is achieved through the implementation of two decision thresholds, enabling segmentation into three distinct intervals. This approach provides finer granularity compared to binary classification.

For instance:

- A value of 350 corresponds to CLOUDY conditions.
- A value of 920 corresponds to SUN conditions.

A limitation of this method lies in the use of fixed thresholds, which do not adapt dynamically to seasonal variations, geographic differences, or sensor aging, potentially affecting classification accuracy.

12.3 System Decision Logic

The system control logic activates solar tracking only when ambient light exceeds a predefined threshold. Otherwise, the servomotors maintain their last known position.

This design decision ensures:

- Reduced mechanical wear
- Lower energy consumption
- Elimination of unnecessary oscillations (jitter)

The integration of algorithm-based positioning with real sensor validation results in a hybrid control strategy that improves system robustness and operational efficiency.



12.4 Data Structure Definition

The implementation of a structured data type enables the grouping of heterogeneous variables into a single logical entity.

The defined structure encapsulates time and solar position parameters, improving code readability, modularity, and maintainability.

This approach is conceptually equivalent to a *record* in Pascal, with similar access semantics.

12.5 Array of Structures (Solar Table)

The solar dataset is implemented as an array of structured elements, each representing a solar position at a specific time.

Each entry includes:

- Hour
- Minute
- Azimuth
- Altitude

This organization allows efficient storage and retrieval of time-indexed solar data while maintaining clarity and scalability.

The use of precomputed values eliminates the need for real-time trigonometric calculations, which is advantageous given the limited computational resources of the Arduino platform.

12.6 Solar Position Lookup Algorithm

The function responsible for selecting the appropriate solar position operates by minimizing the temporal difference between the current time and stored dataset entries.

An iterative search mechanism evaluates all entries and identifies the closest match in terms of total elapsed minutes.

This method ensures that a valid solar position is always available, even when an exact match is not present.

12.7 Conversion to Servo Angles

Solar coordinates (azimuth and altitude) are not directly compatible with servo actuation ranges and therefore require proportional mapping.

The `map()` function performs this transformation, converting solar data into servo-compatible angular positions.



Additionally, the use of rounding techniques improves positional accuracy, while constraints ensure that output values remain within safe mechanical limits.

Direct use of raw solar values would lead to invalid commands, unpredictable behavior, and potential mechanical damage.

12.8 Conditional Update Strategy

The system employs a conditional update mechanism in which servo commands are issued only when a change in position is detected.

This strategy prevents redundant command transmission and significantly reduces:

- Mechanical wear
- Energy consumption
- System instability

As a result, the panel maintains a stable orientation between updates.

12.9 EEPROM Data Structure

A dedicated data structure is used to store persistent system parameters in EEPROM memory.

This includes:

- Last servo positions
- Light intensity thresholds
- System state

This approach ensures continuity of operation after power interruptions and preserves calibration settings.

12.10 System Integration

The RTC module provides continuous and accurate time reference, which is essential for determining the correct solar position.

The system integrates two distinct categories of data:

- Deterministic data (time-based)
- Stochastic data (sensor-based)

The combination of these data sources enables adaptive and reliable system behavior.

The implementation of a nearest-time matching algorithm removes the need for strict discretization of time intervals, increasing operational precision.



13.Full Source Code

</> C++

// Versão Final - Servos, RTC, LCD 20x4, LDR + EEPROM

```
#include <Arduino.h>
#include <ThreeWire.h>
#include <RtcDS1302.h>
#include <Servo.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <EEPROM.h>

// LCD 20x4
LiquidCrystal_I2C lcd(0x27, 20, 4);

// RTC
ThreeWire myWire(7, 6, 5);
RtcDS1302<ThreeWire> rtc(myWire);

// Servos
Servo servoX;
Servo servoY;

// LDR
#define PINO_LDR A0
int LIMIAR_SOL_FORTE = 800;
int LIMIAR_LUZ_MEDIA = 500;

// EEPROM
#define EEPROM_ADDR 0

struct EEPROM_Data {
    int servoX_pos;
    int servoY_pos;
    int limiar_sol_forte;
    int limiar_luz_media;
    char estado[10];
};

// Estrutura solar
typedef struct {
    int hora;
    int minuto;
    float azimuth;
    float altitude;
} PosicaoSolarHora;
```



```
// Tabela solar
const PosicaoSolarHora tabelaSolar[] = {
    {7, 0, 76.87, 0.93}, {7, 15, 79.33, 3.48}, {7, 30, 81.78,
6.18}, {7, 45, 84.22, 8.93},
    {8, 0, 86.67, 11.71}, {8, 15, 89.13, 14.50}, {8, 30, 91.62, 17.30},
{8, 45, 94.15, 20.10},
    {9, 0, 96.74, 22.89}, {9, 15, 99.40, 25.67}, {9, 30, 102.15, 28.42},
{9, 45, 105.01, 31.15},
    {10, 0, 108.00, 33.84}, {10, 15, 111.15, 36.49}, {10, 30, 114.48,
39.08}, {10, 45, 118.02, 41.59},
    {11, 0, 121.80, 44.03}, {11, 15, 125.85, 46.36}, {11, 30, 130.22,
48.58}, {11, 45, 134.94, 50.65},
    {12, 0, 140.04, 52.55}, {12, 15, 145.53, 54.25}, {12, 30, 151.43,
55.72}, {12, 45, 157.72, 56.93},
    {13, 0, 164.36, 57.84}, {13, 15, 171.26, 58.44}, {13, 30, 178.32,
58.70}, {13, 45, 185.41, 58.61},
    {14, 0, 192.41, 58.18}, {14, 15, 199.19, 57.42}, {14, 30, 205.66,
56.35}, {14, 45, 211.75, 55.01},
    {15, 0, 217.45, 53.42}, {15, 15, 222.74, 51.61}, {15, 30, 227.64,
49.62}, {15, 45, 232.18, 47.48},
    {16, 0, 236.38, 45.20}, {16, 15, 240.30, 42.82}, {16, 30, 243.95,
40.34}, {16, 45, 247.38, 37.79},
    {17, 0, 250.62, 35.17}, {17, 15, 253.68, 32.50}, {17, 30, 256.61,
29.80}, {17, 45, 259.42, 27.05},
    {18, 0, 262.12, 24.29}, {18, 15, 264.75, 21.51}, {18, 30, 267.31,
18.72}, {18, 45, 269.82, 15.92},
    {19, 0, 272.30, 13.13}, {19, 15, 274.75, 10.35}, {19, 30, 277.20,
7.59}, {19, 45, 279.64, 4.87},
    {20, 0, 282.10, 2.23}
};

// ----- EEPROM -----

void salvarEEPROM(int sx, int sy, String estado) {
    EEPROM_Data data;

    data.servoX_pos = sx;
    data.servoY_pos = sy;
    data.limiar_sol_forte = LIMIAR_SOL_FORTE;
    data.limiar_luz_media = LIMIAR_LUZ_MEDIA;

    estado.toCharArray(data.estado, 10);

    EEPROM.put(EEPROM_ADDR, data);
}
```



```
void carregarEEPROM(int &sx, int &sy, int &lsf, int &llm, String &estado)
{
    EEPROM_Data data;
    EEPROM.get(EEPROM_ADDR, data);

    if (data.servoX_pos < 0 || data.servoX_pos > 180) {
        sx = 90;
        sy = 0;
        lsf = 800;
        llm = 500;
        estado = "INIT";
        return;
    }

    sx = data.servoX_pos;
    sy = data.servoY_pos;
    lsf = data.limiar_sol_forte;
    llm = data.limiar_luz_media;
    estado = String(data.estado);
}

// ----- Funções -----

String avaliarEstadoTempo(int valor){
    if(valor > LIMIAR_SOL_FORTE) return "SUN";
    else if(valor > LIMIAR_LUZ_MEDIA) return "PARTLY";
    else return "CLOUDY";
}

PosicaoSolarHora encontrarPosicaoMaisProxima(int hora, int minuto){
    PosicaoSolarHora pos = tabelaSolar[0];
    int menorDif = 10000;

    for(int i=0;i<50;i++){
        int dif = abs((tabelaSolar[i].hora*60 + tabelaSolar[i].minuto) -
(hora*60 + minuto));
        if(dif < menorDif){
            menorDif = dif;
            pos = tabelaSolar[i];
        }
    }
    return pos;
}

// ----- Setup -----

void setup() {
    Serial.begin(9600);
```



```
lcd.init();  
lcd.backlight();  
  
servoX.attach(11);  
servoY.attach(3);  
  
rtc.Begin();  
rtc.SetIsRunning(true);  
  
int sx, sy;  
String estado;  
  
carregarEEPROM(sx, sy, LIMAR_SOL_FORTE, LIMAR_LUZ_MEDIA, estado);  
  
servoX.write(sx);  
servoY.write(sy);  
  
// Tela inicial de apresentação  
lcd.clear();  
lcd.setCursor(4,0); lcd.print("System Ready");  
lcd.setCursor(5,1); lcd.print("Sun Tracker");  
lcd.setCursor(4,2); lcd.print("IEFP Braga");  
lcd.setCursor(2,3); lcd.print("41.5294, -8.4569");  
delay(5000);  
  
}  
  
// ----- Loop -----  
  
void loop() {  
  
    // LDR leitura  
    int valorLDR = 0;  
    for(int i=0;i<10;i++){  
        valorLDR += analogRead(PINO_LDR);  
        delay(5);  
    }  
    valorLDR /= 10;  
  
    // RTC  
    RtcDateTime now = rtc.GetDateTime();  
    int hora = now.Hour();  
    int minuto = now.Minute();  
  
    char horaBuffer[6];  
    sprintf(horaBuffer,"%02d:%02d",hora,minuto);  
}
```



```
int horaAtual = hora*60 + minuto;
int horaInicio = tabelaSolar[0].hora*60 + tabelaSolar[0].minuto;
int horaFim = tabelaSolar[52].hora*60 + tabelaSolar[52].minuto; //Nº
'52' intervalos da tabela (13h x 4 intervalos - 15min em 15 min)

String estado;
float azimutReal = 0;
float altitudeReal = 0;
int servoPosX = 90;
int servoPosY = 0;

if(horaAtual < horaInicio || horaAtual > horaFim){
    estado = "NIGHT";
    servoPosX = 90;
    servoPosY = 0;
} else {
    estado = avaliarEstadoTempo(valorLDR);

    if(estado != "CLOUDY"){
        PosicaoSolarHora pos = encontrarPosicaoMaisProxima(hora,minuto);

        azimutReal = pos.azimuth;
        altitudeReal = pos.altitude;

        servoPosX = map(pos.azimuth, 87, 272, 0, 180);
        servoPosY = map(pos.altitude, 0, 60, 0, 75); //inclinação máxima

        servoPosX = constrain(servoPosX,0,180);
        servoPosY = constrain(servoPosY,0,75);
    }
}

servoX.write(servoPosX);
servoY.write(servoPosY);

// -----Configuração LCD 20x4-----

//----- linha 1-----
lcd.clear();
lcd.setCursor(0,0);
lcd.print("Team: Brunos");

//----- linha 2-----
lcd.setCursor(0,1);
lcd.print("Time: ");
lcd.print(horaBuffer);
```



```
//----- linha 3-----  
lcd.setCursor(0,2);  
lcd.print("Status: ");  
lcd.print(estado);  
  
//----- linha 4-----  
char xyBuffer[20];  
sprintf(xyBuffer,"Az:%03d  
Tilt:%02d",(int)azimutReal,(int)altitudeReal);  
  
lcd.setCursor(0,3);  
lcd.print(xyBuffer);  
  
// EEPROM save  
salvarEEPROM(servoPosX, servoPosY, estado);  
  
// Serial Monitor  
  
Serial.println("----- DEBUG -----");  
  
Serial.print("Hora: ");  
Serial.println(horaBuffer);  
  
Serial.print("LDR: ");  
Serial.println(valorLDR);  
  
Serial.print("Estado: ");  
Serial.println(estado);  
  
Serial.print("Azimuth: ");  
Serial.println(azimutReal);  
  
Serial.print("Altitude: ");  
Serial.println(altitudeReal);  
  
Serial.print("Servo X: ");  
Serial.println(servoPosX);  
  
Serial.print("Servo Y: ");  
Serial.println(servoPosY);  
  
Serial.println("-----");  
  
delay(1000);  
}
```




14. Conclusion

The developed system successfully meets the proposed objectives, demonstrating effective integration of mechanical, electronic, and control subsystems in a compact renewable energy application.

The algorithm-based solar tracking strategy proved to be more stable and reliable than sensor-only approaches, while the integration of sensors and persistent memory increased system robustness.

The project provides a solid foundation for future developments, with strong potential for real-world solar energy applications.